



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO
COMPUTADORA



REPORTE DE PRÁCTICA N° 03

NOMBRE COMPLETO: Cruz Cano Omar

N° de Cuenta: 318068932

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE: 2027 - 1

FECHA DE ENTREGA LÍMITE: 17 de septiembre de 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Crear un cohete espacial con instancias de: pirámides, cilindros, esferas, conos, y cubos.

Primero que nada, hicimos la implementación de un piso de color gris oscuro que nos servirá de referencia para nuestras otras figuras; se trata de un cubo escalado en **X** y **Z** a un valor de 150.0, mientras que en **Y** la escala será de 0.5.

```
// Piso gris oscuro
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, -5.0f, -4.0f));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
model = glm::scale(model, glm::vec3(15.0f, 0.5f, 15.0f));
color = glm::vec3(0.25f, 0.25f, 0.25f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[0]->RenderMesh();
```

Para la construcción de nuestro cohete utilizamos como cuerpo un cilindro el cual escalamos en **Y** para que fuera más alto y le agregamos una punta la cual fue construida a partir de un cono, tanto el cilindro como el cono cuentan con la misma resolución para que encajen perfectamente uno sobre el otro además del radio en 1.0f; el cono de la punta también se encuentra escalado en **Y** en la misma dimensión para darle un aspecto puntiagudo. Para la construcción de las 4 aletas del cohete creamos una pirámide cuadrangular de nombre **ala** cuya base se encuentra en el plano **YZ** para facilitar su manipulación y acomodo.

```
// Ala
void CrearAla()
{
    unsigned int indices_ala[] = {
        0, 2, 1,
        0, 3, 2,
        0, 1, 4,
        1, 2, 4,
        2, 3, 4,
        3, 0, 4
    };

    GLfloat vertices_ala[] = {
        0.0f, 0.5f, 0.125f, //0
        0.0f, 0.5f, -0.125f, //1
        0.0f, -0.5f, -0.125f, //2
        0.0f, -0.5f, 0.125f, //3
        0.5f, -0.5f, 0.0f, //4
    };

    Mesh* ala = new Mesh();
    ala->CreateMesh(vertices_ala, indices_ala, 15, 18);
    meshList.push_back(ala);
}
```

A continuación se muestra la lógica seguida para la implementación de las aletas del cohete, se trabajó en pares y solamente utilizando traslaciones y rotaciones acomodamos las 4 aletas; es importante señalar que las aletas cuentan con una escala en los ejes **X** y **Y** y que están pintadas de un gris más oscuro para lograr diferenciar.

```
// Cohete - Ala 1
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(-6.0f, -2.0f, -9.0f));
model = glm::scale(model, glm::vec3(2.0f, 3.0f, 1.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
color = glm::vec3(0.5f, 0.5f, 0.5f); // Color gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();

// Cohete - Ala 2
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, -1.0f, 0.0f));
model = glm::translate(model, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();
```

Generar la figura de unión de 8 pirámides como la que se adjunta. Cada cara de cada pirámide es de un color diferente considerando que al ver una sola pirámide desde todos los ángulos se verían las caras triangulares de color: roja, verde, amarilla, magenta. Y la cara cuadrada se ve de color azul.

Para la segunda actividad del reporte fue necesario hacerle una modificación al archivo **shadercolor.vert** para asegurarnos que se está multiplicando el vector **view** ya que si no realizamos el cambio la impresión de figuras quedará fija al centro de la pantalla.

```
void main()
{
    gl_Position=projection*view*model*vec4(pos,1.0f);
    vColor=vec4(color,1.0f);
}
```

Para hacer uso del shaderColor solamente hay que mandar llamar los siguientes métodos, los cuales asignan en el uniform correspondiente la localidad de memoria del modelo, proyección y vista; se omite color en este shader.

```
shaderList[1].useShader();
uniformModel = shaderList[1].getModelLocation();
uniformProjection = shaderList[1].getProjectLocation();
uniformView = shaderList[1].getViewLocation();
```

Creamos la función **CrearEnder()** la cual almacena todos los vértices de nuestra pirámide cuadrangular así como la información de color necesaria para que cada cara de nuestra figura fuera de un color diferente. Para poder implementar esta pirámide fue necesario instanciar una **MeshColorList** a la cual añadimos nuestra figura de nombre **Ender** y podremos encontrarlo en el índice 0 de dicha lista.

```
// Ender
void CrearEnder()
{
    GLfloat vertices_ender[] = {
        //X          Y          Z          R          G          B
        // Base azul
        1.0f,  -1.0f,    1.0f,    0.0f,  0.0f,  1.0f,
        -1.0f, -1.0f,    1.0f,    0.0f,  0.0f,  1.0f,
        -1.0f, -1.0f,   -1.0f,    0.0f,  0.0f,  1.0f,
        1.0f,  -1.0f,   -1.0f,    0.0f,  0.0f,  1.0f,
        -1.0f, -1.0f,   -1.0f,    0.0f,  0.0f,  1.0f,
        1.0f,  -1.0f,    1.0f,    0.0f,  0.0f,  1.0f,
        // Cara roja
        0.0f,   1.0f,    0.0f,    1.0f,  0.0f,  0.0f,
        1.0f,  -1.0f,    1.0f,    1.0f,  0.0f,  0.0f,
        -1.0f, -1.0f,    1.0f,    1.0f,  0.0f,  0.0f,
        // Cara verde
        0.0f,   1.0f,    0.0f,    0.0f,  1.0f,  0.0f,
        1.0f,  -1.0f,   -1.0f,    0.0f,  1.0f,  0.0f,
        -1.0f, -1.0f,   -1.0f,    0.0f,  1.0f,  0.0f,
        // Cara magenta
        0.0f,   1.0f,    0.0f,    1.0f,  0.0f,  1.0f,
        -1.0f, -1.0f,   -1.0f,    1.0f,  0.0f,  1.0f,
        -1.0f, -1.0f,    1.0f,    1.0f,  0.0f,  1.0f,
        // Cara amarilla
        0.0f,   1.0f,    0.0f,    1.0f,  1.0f,  0.0f,
        1.0f,  -1.0f,   -1.0f,    1.0f,  1.0f,  0.0f,
        1.0f,  -1.0f,    1.0f,    1.0f,  1.0f,  0.0f,
    };

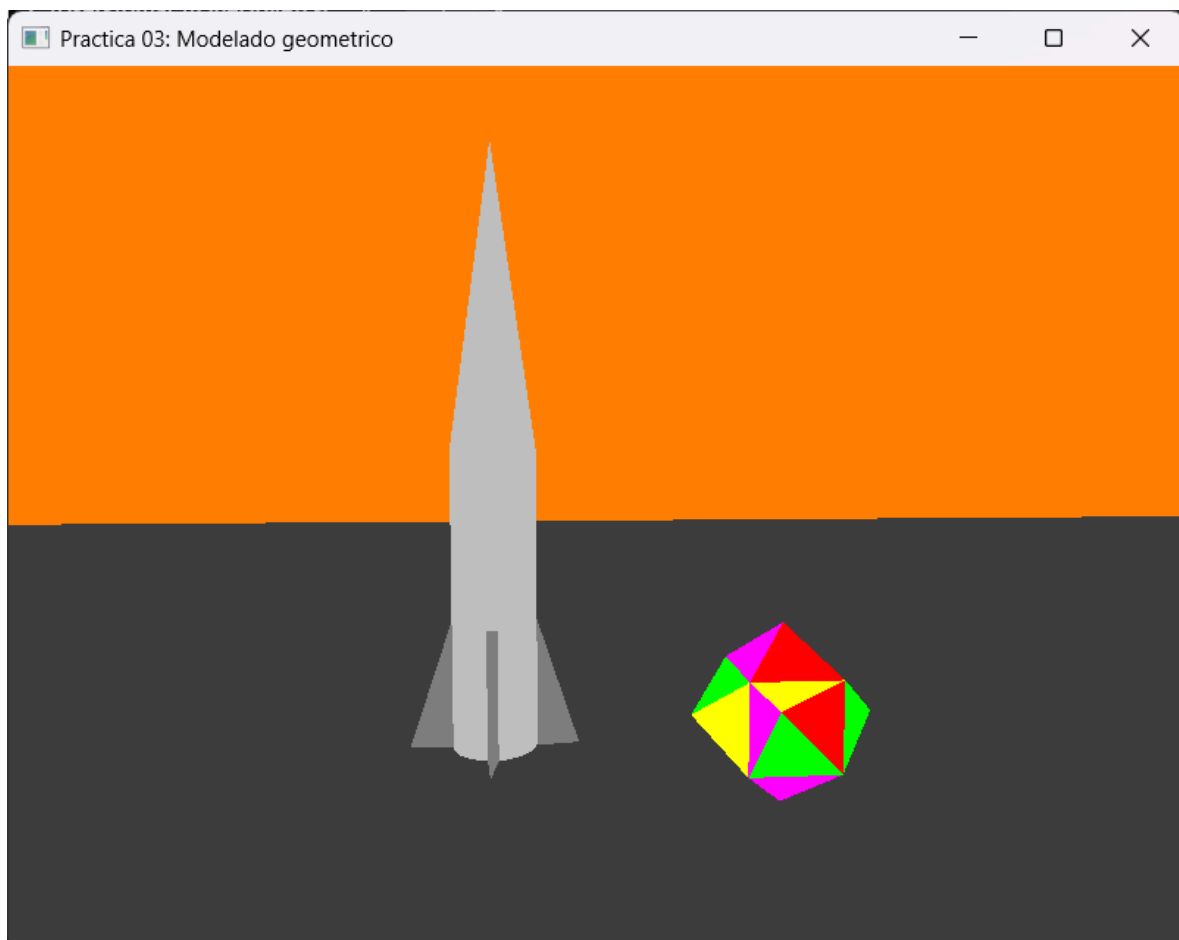
    MeshColor* ender = new MeshColor();
    ender->CreateMeshColor(vertices_ender, 108);
    meshColorList.push_back(ender);
}
```

Se siguió la siguiente lógica para la implementación de las 8 pirámides que conformarán a nuestra figura final. Es muy importante señalar que al trabajar con un **shaderColor** es necesario actualizar las variables uniform correspondientes para que la figura se despliegue correctamente, además debemos de asegurarnos de usar el método **RenderMeshColor()**

```
// Ender
model = glm::mat4(1.0);
model = glm::scale(model, glm::vec3(0.75f, 0.375f, 0.75f));
model = glm::translate(model, glm::vec3(0.0f, 1.0f, -4.0f));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[0]->RenderMeshColor();

// 1
model = glm::translate(model, glm::vec3(0.0f, -2.0f, 0.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[0]->RenderMeshColor();
```

Se mantiene el fondo de color naranja y se hicieron pequeñas modificaciones al **Window.cpp** para cambiar el nombre de la ventana.



Conclusión

Las actividades de la práctica permitieron afianzar el modelado geométrico tridimensional mediante la combinación y acoplamiento de primitivas como cubos, cilindros, conos y pirámides. La mayor complejidad técnica radicó en sincronizar los parámetros geométricos como radios y niveles de resolución entre el cuerpo y la punta del cohete, además de manipular con precisión las matrices para ensamblar el arreglo de ocho pirámides y adaptar el vertex shader con la matriz de vista para renderizar correctamente los colores por vértice sin que la escena quede estática.

La segunda actividad reforzó conocimientos revisados en sesiones anteriores ya que hicimos uso del MeshColor y su correspondiente shader para lograr imprimir una figura pero con todas las caras de un color diferente, lo que requirió definir con precisión los atributos de color por cada vértice en el VAO y actualizar debidamente las matrices de modelo, vista y proyección mediante las variables uniform.